Day - 23

NumPy Arrays & Data Cleaning

in Python

SkillCourse

E-learning Platform by Satish Dhawale

A.I. Powered 30 Days Python Micro Course By Satish Dhawale (Microsoft Certified Trainer)

NumPy stands for **Numerical Python.**

- It is the **core library** for numerical and scientific computing in Python.

- NumPy is the foundation of all numerical operations in Data Analysis, Machine Learning, and even Pandas internally uses NumPy.

- High-performance **multi-dimensional arrays**
- Fast numerical computations
- Vectorized mathematical operations
- Foundation for Pandas, SciPy, Scikit-learn, TensorFlow, PyTorch

SkillCourse

Who Created NumPy?

NumPy was originally developed by:

👤 **Jim Hugunin (1995)**

He created a package called **Numeric**, the first numerical extension for Python.

It introduced multi-dimensional arrays for the first time.

Later...

👤 **Travis Oliphant (2005)**

He rewrote and improved Numeric

Added features from another library called **Numarray**

Combined both into a single powerful library called: **NumPy**

Travis Oliphant is considered **the main creator of modern NumPy**.

SkillCourse

Why Was NumPy Created?

Before NumPy:

Python was slow for numerical calculations

Python lists took too much memory

No proper array structure

No fast mathematical operations

Data scientists relied on MATLAB or R

Scientific computing had no standard library in Python

NumPy was created to solve **three main problems**:

1. Speed

- NumPy arrays are stored in **contiguous memory**, unlike Python lists. This gives **C-like performance** inside Python.
- NumPy is written in **C and C++** internally; therefore:
- NumPy is **100x faster** than Python lists for numerical operations.

2. **Efficiency** (Low Memory)

A Python list:

- Stores data as full Python objects
- Needs extra memory for type info & pointers
- NumPy arrays:
- Store data in one compact block
- Use less memory
- Use fixed data types
- Perfect for large datasets

Vectorized Operations

- NumPy allows you to perform mathematical operations **without loops**.

  Example:

  arr * 10

- This operation runs in **compiled C code**, not in Python.
  This makes NumPy incredibly fast.

NumPy vs Python Lists

Feature	Python List	NumPy Array
Speed	Slow	Very Fast
Memory	High	Low
Data Type	Mixed allowed	Single fixed type
Operations	Loop needed	Vectorized
Suitable for	Small tasks	Big data / ML / AI

Why Data Analysts Must Learn NumPy First

NumPy teaches:

- Array manipulation
- Indexing, slicing
- Vectorized operations
- Mathematical functions
- Data cleaning logic used later in Pandas
- If NumPy is clear → Pandas becomes incredibly easy.

SkillCourse

Real Industries Using NumPy

- Finance (Risk models, forecasting)
- E-commerce (Recommendation systems)
- Healthcare (Medical imaging)
- Manufacturing (Sensor analysis, IoT)
- Research & Academia
- Machine Learning & Deep Learning companies

SkillCourse

What is NumPy?

Layman Explanation

- NumPy is like **Excel + Calculator + Super Speed** — all in one.
- Works with large data faster than lists
- Performs mathematical operations very fast
- Used in Pandas, ML, AI, and dashboards

Technical Definition

- NumPy = Numerical Python
  It provides high-performance **multi-dimensional arrays** and **vectorized mathematical operations**.

SkillCourse

Installing & Importing NumPy

```
pip install numpy
import numpy as np
```

Creating NumPy Arrays : Create 1D Array

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr)
```

Create Array

Create 2D Array

- arr2 = np.array([[10, 20], [30, 40]])
- print(arr2)

Create Array with Range

- arr = np.arange(1, 11)
- print(arr)

Array with Fixed Values

- zeros = np.zeros(5)
- ones = np.ones(5)

SkillCourse

Array

Random Array (Very Useful)

- rand = np.random.randint(10, 100, 5)
- print(rand)

Array Indexing

- arr = np.array([10, 20, 30, 40, 50])
- print(arr[0]) # 10
- print(arr[-1]) # 50

Slicing

```
print(arr[1:4])   # [20 30 40]
print(arr[:3])    # [10 20 30]
print(arr[2:])    # [30 40 50]
```

Vectorized Operations

Addition

```
arr = np.array([10, 20, 30])
print(arr + 10)
```

Multiply

```
print(arr * 2)
```

Square of all items

```
print(arr ** 2)
```

Array Functions (Data Cleaning + Analysis)

✓ **Sum, Mean, Min, Max**

```
arr = np.array([10, 20, 30])

print(arr.sum())
print(arr.mean())
print(arr.min(), arr.max())
```

✓ **Standard Deviation**

```
print(arr.std())
```

✓ **Reshape Array**

```
arr = np.arange(1, 10)
print(arr.reshape(3,3))
```

Real Data Cleaning Examples

Example 1 – Remove Negative Values

```
data = np.array([10, -20, 30, -40, 50])
clean = data[data >= 0]
print(clean)
```

Example 2 – Replace Missing Values (-1 means missing)

```
marks = np.array([80, 90, -1, 75, -1, 60])

marks[marks == -1] = marks[marks != -1].mean()

print(marks)
```

Real Data Cleaning Examples

Example 3 – Filter Values > 20

```
arr = np.array([10, 25, 30, 5, 50])
filtered = arr[arr > 20]
print(filtered)
```

Example 4 – Multiply All Prices by Qty (Vectorized Calculation)

```
qty = np.array([1,2,1,3])
price = np.array([100,200,300,400])

total = qty * price
print(total)
```

SkillCourse